PRACTICAL –10

Aim: Implement Kruskal's Algorithm.

Algorithm:

input: No. of vertices, No. of edges.
output: edges in MST & min cost.
step-1: start
step-2: Read the no. of vertices & edges.
step-3: State all edges with this weights.
step-4: Sort the edges in ascending order of weight
step-5: Select the edge with the min weight.
step-6: Check whether adding the edge form a cycle.
step-7: If it does not form a cycle, add the edges to the MST
step-8: Repeat steps 5-7 until V-1 edges are selected, when V is the no. of vertices.
step-9: Display the selected edges and the total min cost.
step-10: stop.

Progarm:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Edge {
```

```
    int u, v, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

int findParent(int parent[], int i) {
    if (parent[i] == i)
        return i;
    return findParent(parent, parent[i]);
}

int main() {
    int n, e;
    Edge edges[100];

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter number of edges: ";
    cin >> e;
```

```
cout << "Enter edges (source destination weight):\n";

for (int i = 0; i < e; i++) {
    cin >> edges[i].u >> edges[i].v
        >> edges[i].weight;
}

sort(edges, edges + e, compare);

int parent[100];

for (int i = 1; i <= n; i++) {
    parent[i] = i;
}

int minCost = 0;
int count = 0;

cout << "\nEdges in Minimum Spanning Tree:\n";

for (int i = 0; i < e && count < n - 1; i++) {
```

```
        int u = findParent(parent, edges[i].u);
        int v = findParent(parent, edges[i].v);

        if (u != v) {
            cout << edges[i].u << " - "
                 << edges[i].v << " : "
                 << edges[i].weight << endl;

            minCost += edges[i].weight;
            parent[u] = v;
            count++;
        }
    }

    if (count != n - 1) {
        cout << "Graph is disconnected.\n";
    } else {
        cout << "Minimum Cost = " << minCost << endl;
    }

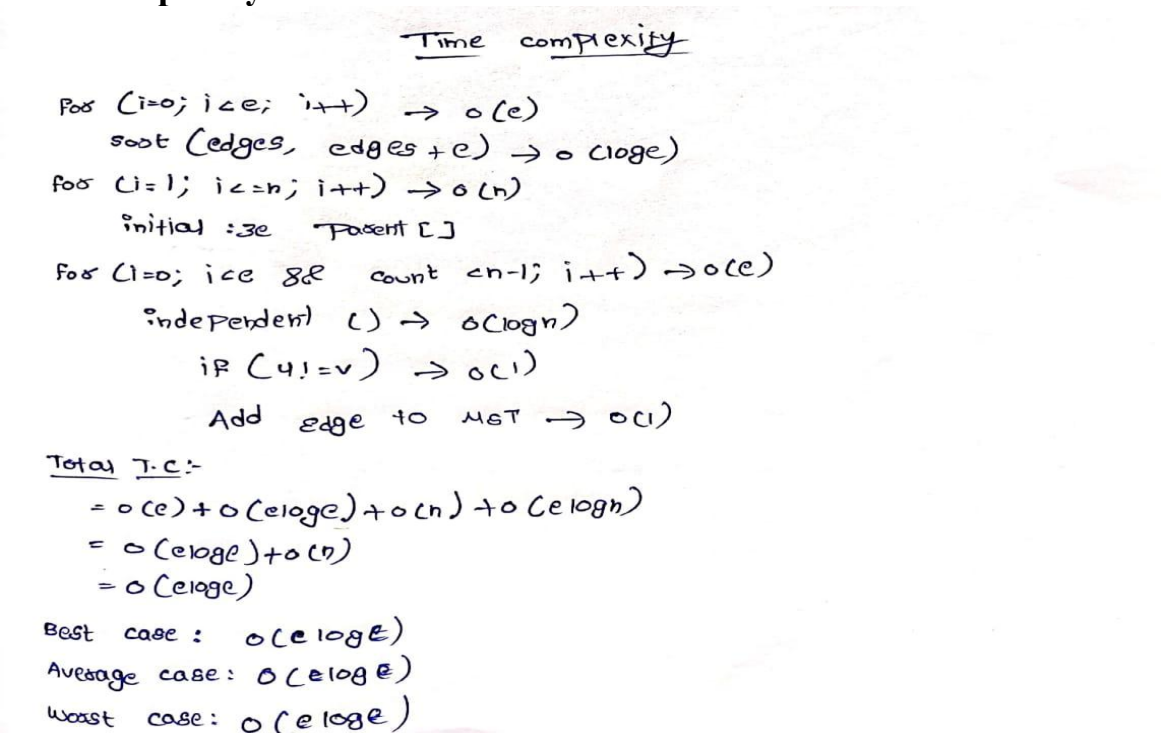
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter the adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

Edges in Minimum Spanning Tree:
1 - 2 : 2
2 - 3 : 3
3 - 4 : 5
Minimum Cost = 10
```

Time complexity:



Conclusion: Kruskal's algorithm has a time complexity of (O(Elog E), mainly because it sorts all edges in ascending order of their weights.